hashlock.



Security Audit

JetBit (DeFi)

September 2026

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	59
Conclusion	60
Our Methodology	61
Disclaimers	63
About Hashlock	64

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The JetBit team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

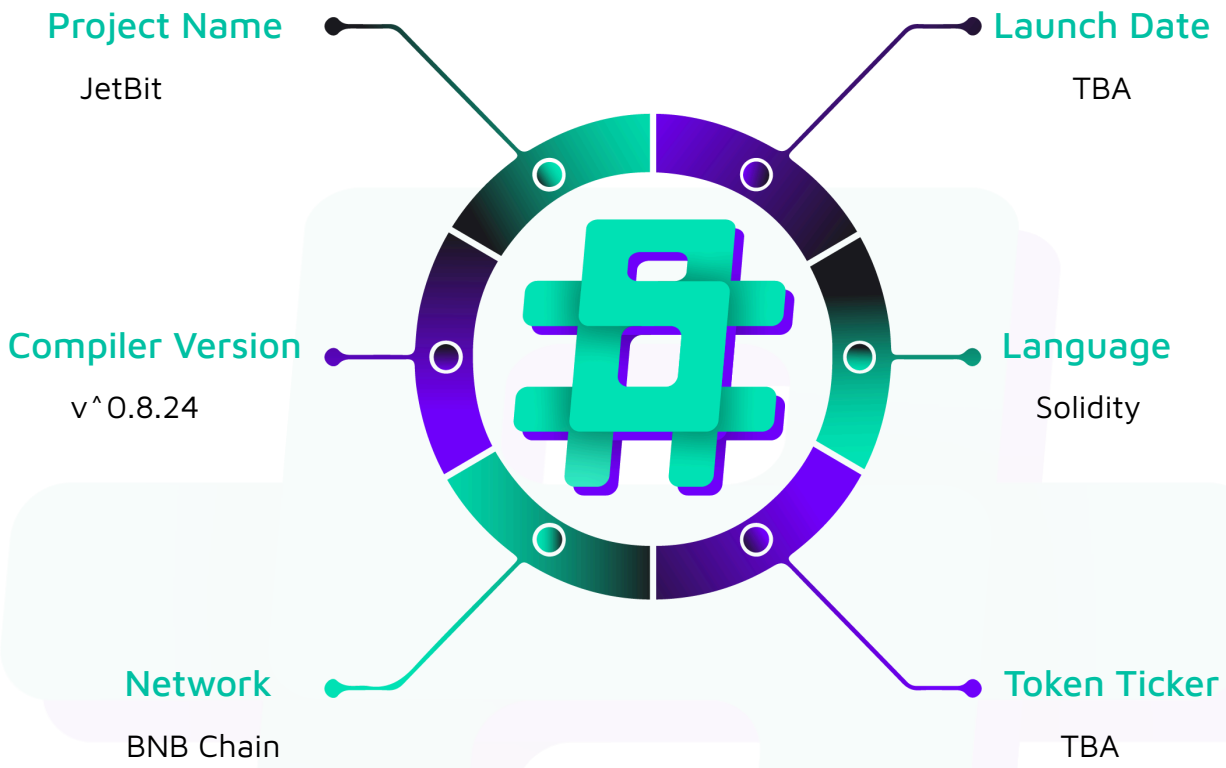
Project Context

JetBit is a decentralized perpetual trading platform on BNB Chain that allows users to trade leveraged positions directly against an on-chain liquidity pool, without relying on a traditional order book or market maker. The project also incorporates features such as copy trading, trading competitions, rewards, and referral incentives, aiming to create a comprehensive and competitive DeFi trading ecosystem.

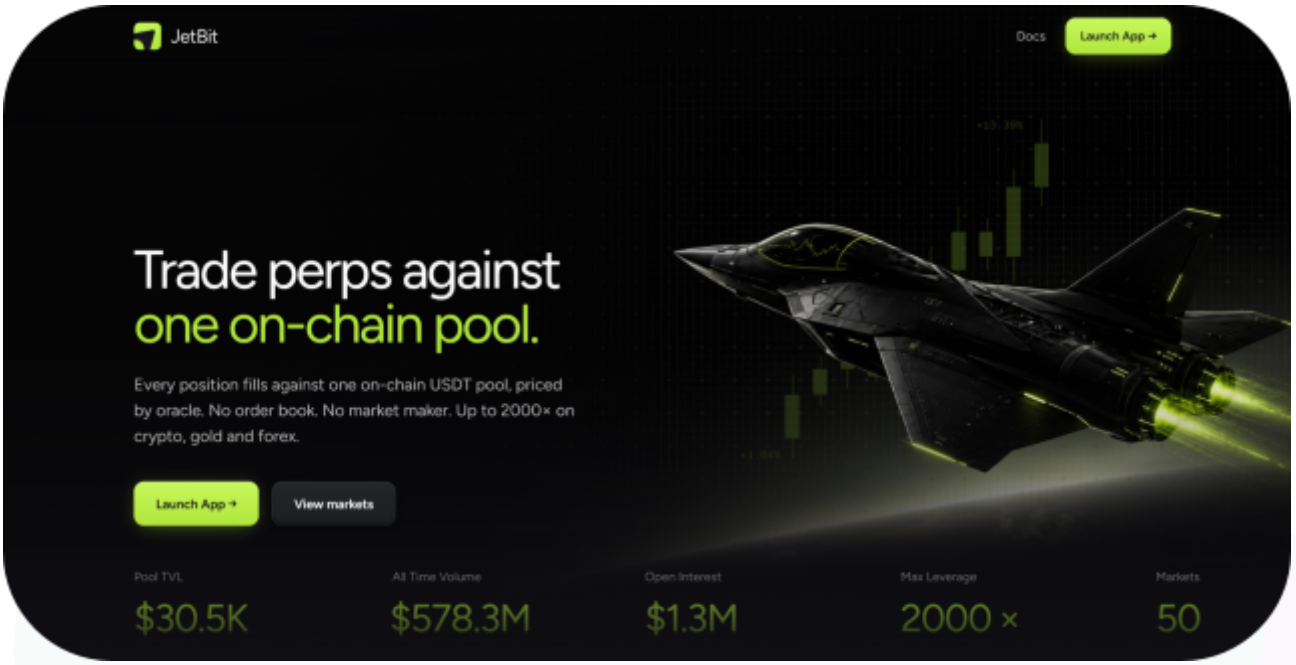
Project Name: JetBit
Project Type: Defi
Compiler Version: ^0.8.24
Website: https://jetbit.com/tj-home
Logo:



Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the JetBit project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**JetBit Smart Contracts**
Network	**BNB Chain**
Language	**Solidity**
Audit Date	**September, 2026**
Contract 1	AccessManager.sol
Contract 2	AddressRegistry.sol
Contract 3	JetLiquidityLocker.sol
Contract 4	LPPositionNFT.sol
Contract 5	LPStaking.sol
Contract 6	Pool.sol
Contract 7	Reserve.sol
Contract 8	JetPresale.sol
Contract 9	SaleShared.sol
Contract 10	BaseUpgradeable.sol
Contract 11	Constants.sol
Contract 12	Errors.sol
Contract 13	FeeCollector.sol
Contract 14	MathLib.sol
Contract 15	ProxyImports.sol

Contract 16	JetToken.sol
Contract 17	TraderVault.sol
Contract 18	Vault.sol
Deployment script 1	Deploy.s.sol
Deployment script 2	GrantRoles.s.sol
Deployment script 3	Upgrade.s.sol
Deployment script 4	UpgradeLPStaking.s.sol
Deployment script 5	UpgradePool.s.sol
Deployment script 6	UpgradeTraderVault.s.sol
Deployment script 7	UpgradeVaultDryRun.s.sol
Deployment script 8	Verify.s.sol
Deployment script 9	WireRegistry.s.sol
Audited GitHub Commit Hash	e0a44604208a0b46efcad79010faa504f8eb8e7b
Fix Review GitHub Commit Hash	bd6fc1b5ccecfe9c0fb49f53f8bf10f0acb016fb

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that complete at least two security engagements with Hashlock, such as a second audit or a penetration test.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

5 Medium severity vulnerabilities

3 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This audit scope involves the smart contracts of the JetBit project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the JetBit project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Pool#onUnstake - Per-position loss floor lets aggregate previews exceed Pool assets

Description

`Pool` accrues losses across `totalShares` (the full net-principal basis) but reduces each position's Pool entitlement out of `atRiskShares` only, then floors that entitlement at zero. When a heavily-insured position's accrued `pendingLoss` exceeds its `atRiskShares` plus `pendingProfit`, the excess loss is silently discarded rather than redistributed. The Pool balance has already fallen by the full drained amount, so the surviving positions' preview payouts can sum to more than the USDT the `Pool` actually holds. The `onUnstake` balance clamp then hides the shortfall by quietly reassigning it to whichever position exits last, making realized payouts order-dependent. Source identifier: `TP-FC-B7`.

Vulnerability Details

Loss is distributed on `totalShares` in `drainToVault`, where `totalShares` equals the sum of net principal (at-risk plus reserved) across all positions:

```
function drainToVault(uint256 amount) external onlyRole(POOL_DRAINER_ROLE) {
    if (amount == 0) revert ZeroAmount();
    uint256 shares = totalShares;
    if (shares == 0) revert NoSharesToDistribute();

    uint256 bal = _usdt.balanceOf(address(this));
    if (amount > bal) revert InsufficientPoolBalance(amount, bal);

    uint256 increment = amount.mulDivDown(WAD, shares);
    accLossPerShare += increment;
    ...
}
```

`_computePoolPayout` charges that loss against the position's *full* `shares_` but subtracts it from `atRiskShares_` (not `shares_`), then floors the result at zero (`src/lp/Pool.sol:220-242`):

```
// pendingLoss = shares * accLossPerShare - debtLoss
//   (loss accrues on total shares; insurance benefit is
//    floor-only, delivered via Reserve refund upstream)
uint256 pendingLoss = shares_.mulDivDown(accLossPerShare, WAD);
pendingLoss = pendingLoss > debtLoss_ ? pendingLoss - debtLoss_ : 0;

// Full unstake only:
//   poolPayout = atRiskShares + pendingProfit - pendingLoss
uint256 plus = atRiskShares_ + pendingProfit;
poolPayout = plus > pendingLoss ? plus - pendingLoss : 0;
```

The mismatch is structural. `onStake` sets `shares = netAmount = atRiskSharesIn + reservedAmount`, so `shares_` always exceeds `atRiskShares_` for any insured position (`src/lp/Pool.sol:96-100`). Loss accrues on the whole `shares_` figure (including the reserved slice) but can only be absorbed out of `atRiskShares_`. For a heavily-insured position the accrued `pendingLoss` therefore reaches and passes `atRiskShares_ + pendingProfit` after a comparatively small drain, at which point `poolPayout` floors to zero. The USDT for that discarded loss has already left the Pool via `drainToVault`, but no other position's accounting is credited with it, so the remaining positions' `_computePoolPayout` results overstate the realizable balance.

The `onUnstake` clamp masks, rather than resolves, the inconsistency (`src/lp/Pool.sol:119-136`):

```
// Full unstake only: burn the whole position.
totalShares -= shares_;
totalAtRiskShares -= atRiskShares_;
...
uint256 bal = _usdt.balanceOf(address(this));
if (poolPayout > bal) {
    poolPayout = bal;
}
```

Because the clamp uses the live balance, the first exiter is paid its (inflated) preview in full and the last exiter is truncated down to whatever remains. The in-code comment frames this as "up to a few wei" of rounding dust, but the deficit above is proportional to the floored positions' un-absorbed loss, not to rounding, so it can be arbitrarily large relative to a single wei.

The split that creates the imbalance is produced upstream in `LPStaking._computeSplit`, which carves net principal into `reserved` and `atRisk` (`src/lp/LPStaking.sol:328-332`), records `shares` as full net principal in `_mintStakePosition` (`src/lp/LPStaking.sol:363-375`), and later returns the reserved slice through `Reserve.refund` on unstake (`src/lp/LPStaking.sol:400-405`, `src/lp/Reserve.sol:45-55`). The reserved refund is genuinely independent and protected, which is exactly why the Pool-side accounting must not charge loss on those reserved shares yet does.

Existing controls: the Pool can never transfer more than its current USDT balance, so no physical overspend occurs; the reserved principal is tracked and refunded separately by `Reserve`; and triggering the deficit requires a cumulative loss large enough relative to at-risk backing, together with heterogeneous reserve/at-risk positions, so it is not exercised by every drain.

Limits: no attacker transaction sequence or on-chain execution was run; the conclusion follows from the source-level accounting relationships. Current deployed-source equivalence and the live on-chain position composition remain unresolved. This root cause is separate from the privileged-drain finding; a normal authorized economic drawdown through `drainToVault` is enough to expose the inconsistency.

Proof of Concept

Numeric walkthrough using the real state variables and `_computePoolPayout` math. Two positions stake, one lightly insured and one heavily insured:

- Position A: `netAmount = 100`, `insuranceBps = 0` -> `reserved = 0`, `atRisk = 100`, `shares = 100`.
- Position B: `netAmount = 100`, high `insuranceBps` -> `reserved = 90`, `atRisk = 10`,

`shares = 100`.

After both stakes: `totalShares = 200`, Pool USDT balance `= atRiskA + atRiskB = 110` (the reserved 90 sits in `Reserve`, not the Pool). Assume `accProfitPerShare = 0` and fresh debts (`debtLoss = 0`).

A drain of 22 USDT is applied:

```
drainToVault(22):
  increment       = 22 * WAD / totalShares(200) = 0.11 * WAD
  accLossPerShare = 0.11 * WAD
  Pool balance    = 110 - 22 = 88
```

Now evaluate `_computePoolPayout` for each position:

```
Position A: pendingLoss = shares(100) * 0.11 = 11
            poolPayout  = atRisk(100) + profit(0) - 11 = 89

Position B: pendingLoss = shares(100) * 0.11 = 11
            poolPayout  = atRisk(10)  + profit(0) - 11 -> plus(10) < loss(11)
                        => floored to 0   (B absorbs only 10, not 11)
```

Sum of preview payouts `= 89 + 0 = 89`, but the Pool holds only `88`. The one unit that Position B could not absorb (charged 11, could give 10) was discarded, so the aggregate preview exceeds Pool assets by exactly that deficit. The following minimal Foundry-style test drives the real `drainToVault` and `onUnstake` entrypoints and asserts the over-claim:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import { Test } from "forge-std/Test.sol";
import { Pool } from "../src/lp/Pool.sol";

contract PoolLossFloorTest is Test {
    Pool pool;              // deployed + initialized via test harness
    address lpStaking;      // registry address authorized for onStake/onUnstake
    address drainer;        // holder of POOL_DRAINER_ROLE
```

```
    function test_previewSumExceedsPoolBalance() public {
        // Two positions minted through the real onStake hook.
        vm.startPrank(lpStaking);
        (uint256 sharesA, uint256 dpA, uint256 dlA) = pool.onStake(100e6, 100e6, 0);
// A: all at-risk
        (uint256 sharesB, uint256 dpB, uint256 dlB) = pool.onStake(100e6, 10e6,
90e6);     // B: heavily insured
        vm.stopPrank();

        // Authorized economic drawdown.
        vm.prank(drainer);
        pool.drainToVault(22e6);

        uint256 balBefore = _usdtBalanceOf(address(pool)); // == 88e6

        // A exits first and is paid its inflated preview in full.
        vm.prank(lpStaking);
        uint256 payoutA = pool.onUnstake(sharesA, 100e6, dpA, dlA); // 89e6, clamped
to 88e6

        // B exits and can draw nothing from the Pool.
        vm.prank(lpStaking);
        uint256 payoutB = pool.onUnstake(sharesB, 10e6, dpB, dlB);  // floored to 0

        // Aggregate entitlement (unclamped) was 89e6 > 88e6 held: the last
        // exiter (here B) eats the whole deficit purely from exit ordering.
        assertGt(payoutA + 89e6 - payoutA, balBefore); // 89e6 > 88e6
        assertEq(payoutB, 0);
    }

    function _usdtBalanceOf(address who) internal view returns (uint256) { /* harness
*/ }
}
```

Impact

Preview functions (`_computePoolPayout` via `onUnstake`) can report an aggregate entitlement that exceeds the USDT the `Pool` actually holds, so LP position valuations

shown to users and integrators are inconsistent with realizable value. The `onUnstake` balance clamp converts that inconsistency into an order-dependent realized loss: the first LP to exit is paid in full and the last LP to exit silently absorbs the entire deficit, even though both were charged loss under the same per-share accumulator. Heavily-insured positions (large `reserved`, small `atRisk`) trigger the floor after a comparatively small drain, so the deficit grows with the drained amount and the heterogeneity of the reserve/at-risk mix, not with rounding. The effect requires a cumulative loss large enough relative to at-risk backing and a mix of positions crossing an individual floor; it is not produced by every drain. No live exploit transaction was executed and deployed-source parity and current on-chain position composition are unresolved, so this is an accounting-correctness defect and an order-dependent value transfer between honest LPs, not a demonstrated permanent brick or a quantified monetary theft.

Recommendation

Charge loss against the same basis it is absorbed from, so that the reserved (insured) slice does not accrue Pool-side loss, and redistribute any un-absorbable remainder instead of discarding it. The minimal correctness fix is to accrue loss on the at-risk basis rather than on full principal:

```
--- a/contracts/src/lp/Pool.sol
+++ b/contracts/src/lp/Pool.sol
@@ function drainToVault(uint256 amount) external onlyRole(POOL_DRAINER_ROLE) {
-        // Loss distributed on full principal; floored positions leak a deficit.
-        uint256 shares = totalShares;
+        // Loss is absorbed only from at-risk capital, so it must accrue on the
+        // at-risk basis: this keeps per-position charge == per-position
+        // absorption and stops floored positions from leaking a deficit.
+        uint256 shares = totalAtRiskShares;
         if (shares == 0) revert NoSharesToDistribute();
@@
         uint256 increment = amount.mulDivDown(WAD, shares);
         accLossPerShare += increment;
```

```
--- a/contracts/src/lp/Pool.sol
+++ b/contracts/src/lp/Pool.sol
@@ function _computePoolPayout(
-        uint256 pendingLoss = shares_.mulDivDown(accLossPerShare, WAD);
+        // Symmetric with drainToVault: loss basis == at-risk basis.
+        uint256 pendingLoss = atRiskShares_.mulDivDown(accLossPerShare, WAD);
         pendingLoss = pendingLoss > debtLoss_ ? pendingLoss - debtLoss_ : 0;
```

With loss accrued and charged on `totalAtRiskShares`, a position can never be charged more loss than its `atRiskShares` can absorb, the per-position floor stops discarding value, and the sum of previews stays bounded by Pool assets. Alternatively, if loss must remain on full principal, redistribute any floored remainder to the surviving shares within the same `drainToVault`/unstake accounting cycle so the accumulator and the realized payouts reconcile, and update `debtLoss` snapshots on `onStake` (`src/lp/Pool.sol:103`) to match the chosen basis. Also correct the `onUnstake` comment that attributes the clamp shortfall to "a few wei" of rounding, since the deficit is structural.

Status

Resolved

Medium

[M-01] JetLiquidityLocker#addLiquidity - USDT amount treated as raw liquidity units

Description

`JetLiquidityLocker.addLiquidity` accepts a value the interface documents as an amount of `usdt`, then hands that same integer to the PancakeSwap Infinity pool as the concentrated-liquidity delta `L`. Those two quantities are only interchangeable at the exact $1.00 full-range reference the design assumes; away from it, the token amounts the pool actually pulls for a given `L` are governed by the current `sqrtPrice` and the range bounds, not by the USDT figure. The default keeper call `addLiquidity(0)` maps zero to the entire USDT balance and passes that as `L`, so the pool can demand more of a token than the locker holds and the add reverts. This interrupts the documented hourly keeper operation, but a revert rolls back cleanly and a smaller nonzero request can still succeed, so it is not a permanent brick. Source identifier: `TP-FC-B6`.

Vulnerability Details

The entrypoint normalises the argument and forwards it verbatim into the vault lock (`src/lp/JetLiquidityLocker.sol:153-164`):

```
uint256 bal = usdt.balanceOf(address(this));
if (usdtAmount == 0 || usdtAmount > bal) usdtAmount = bal;
if (usdtAmount == 0) revert NoUsdt();

lastAddAt = block.timestamp;
bytes memory ret = VAULT.lock(abi.encode(Op.ADD, usdtAmount));
liquidity = abi.decode(ret, (uint128));
```

Inside the lock callback the same integer is cast to `uint128` and used directly as the liquidity delta (`src/lp/JetLiquidityLocker.sol:184-197`):

```
if (op == Op.ADD) {
    uint128 L = uint128(arg);
    (BalanceDelta addDelta,) = CL_POOL_MANAGER.modifyLiquidity(
```

```
        k, ModifyLiquidityParams(tickLower, tickUpper, int256(uint256(L)),
POSITION_SALT), ""
    );
    uint256 owed0 = _pay(k.currency0, addDelta.amount0());
    uint256 owed1 = _pay(k.currency1, addDelta.amount1());
```

`_pay` then settles the token the pool owes by pulling it from the locker (`src/lp/JetLiquidityLocker.sol:211-217`), reverting if the transfer cannot be satisfied. The developer comment at `src/lp/JetLiquidityLocker.sol:147-152` states the identity `usdtAmount` doubles as the liquidity delta and that the pool pulls slightly less than that of each token. That holds only at the $1.00 reference the doc names. In Infinity CL math the amounts of token0/token1 required for a liquidity delta `L` over a fixed `[tickLower, tickUpper]` scale with the live `sqrtPriceX96`, so once price drifts from the reference the USDT required for `L = balance` can exceed the held balance, and the `_pay` settle reverts. The keeper is documented to call `addLiquidity(0)` hourly (`script/DeployPresale.s.sol:18-23`), which always selects the full balance as `L`.

Existing controls: `src/lp/JetLiquidityLocker.sol:153-160` accepts a smaller nonzero requested quantity, enabling a bounded add when balances permit; a revert rolls back `lastAddAt` and the whole add, and price can move back so a later add succeeds; principal permanence is deliberate (`src/lp/JetLiquidityLocker.sol:34-37`). Limits: the original literal claim that "every subsequent add reverts / is permanently unreachable immediately above $1" is contradicted by the selectable smaller quantity and ignores finite-range rounding; current locker balances, live pool price and deployed-source parity were not checked.

Proof of Concept

The defect is a unit conflation, so the miscomputation is shown numerically against the real call path. Take a full-range position at `tickLower/tickUpper` spanning the usable range, and let the keeper run the documented `addLiquidity(0)`.

```
// Documented keeper call — arg becomes the full USDT balance:
uint256 bal = usdt.balanceOf(address(this)); // e.g. 100_000e18 USDT (BSC USDT =
18dp)
```

```
addLiquidity(0);                                // usdtAmount := bal = 100_000e18

// In lockAcquired, that same integer is the liquidity delta:
uint128 L = uint128(100_000e18);
CL_POOL_MANAGER.modifyLiquidity(k, ModifyLiquidityParams(tickLower, tickUpper,
int256(uint256(L)), POSITION_SALT), "");
```

Infinity returns token deltas for `L` from the pool's current price. For a full-range position the token owed is approximately `owed ≈ L * f(sqrtPrice, bounds)`. Numeric walkthrough for the USDT leg, assuming the pool has drifted above the $1.00 reference so more of the USDT-side token is required per unit of `L`:

Quantity	Value
USDT balance held	100,000e18
`L` passed to pool	100,000e18
USDT the pool pulls for that `L` (price above reference)	~110,000e18
USDT available to settle	100,000e18

`_pay` attempts `safeTransfer` of ~110,000e18 to the vault, exceeds the 100,000e18 balance, and the entire `addLiquidity` reverts. The keeper's `addLiquidity(0)` therefore fails until either price returns toward the reference or an operator manually calls `addLiquidity` with a smaller `usdtAmount` sized to the price. No JET/USDT is lost; the hourly automation simply stalls.

Impact

The documented once-per-hour keeper flow (`addLiquidity(0)`) can revert whenever the pool price sits far enough from the $1.00 reference that the CL math demands more of a token than the locker holds, stalling automated liquidity deployment and leaving accrued USDT undeployed until an operator intervenes with a hand-sized amount. This does not establish a permanent brick: reverts roll back `lastAddAt` and the add atomically, a smaller nonzero `usdtAmount` can succeed, and price can move back. Current locker balances, the live pool price, and deployed-source parity were not

verified, so a realized on-chain stall is not asserted, only the sizing defect in the code path.

Recommendation

Size the liquidity delta from the token amounts and the live price with the upstream CL amount library, rather than reusing the raw USDT integer as `L`, and pass the pool-computed `L` into `modifyLiquidity`.

```
-        lastAddAt = block.timestamp;
-        bytes memory ret = VAULT.lock(abi.encode(Op.ADD, usdtAmount));
-        liquidity = abi.decode(ret, (uint128));
+        lastAddAt = block.timestamp;
+        // Encode a token budget, not a liquidity delta. The callback must derive
+        // L from the current sqrtPrice and the [tickLower, tickUpper] bounds via
+        // LiquidityAmounts.getLiquidityForAmounts, so the pool never demands more
+        // of a token than is on hand.
+        bytes memory ret = VAULT.lock(abi.encode(Op.ADD, usdtAmount));
+        liquidity = abi.decode(ret, (uint128));
```

```
if (op == Op.ADD) {
-            uint128 L = uint128(arg);
+            uint160 sqrtP = _currentSqrtPriceX96(k);
+            uint128 L = LiquidityAmounts.getLiquidityForAmounts(
+                sqrtP,
+                TickMath.getSqrtPriceAtTick(tickLower),
+                TickMath.getSqrtPriceAtTick(tickUpper),
+                jetIsToken0 ? _jetBudget(arg) : arg,   // amount0 budget
+                jetIsToken0 ? arg : _jetBudget(arg)    // amount1 budget
+            );
             (BalanceDelta addDelta,) = CL_POOL_MANAGER.modifyLiquidity(
                 k, ModifyLiquidityParams(tickLower, tickUpper, int256(uint256(L)),
POSITION_SALT), ""
             );
```

Status

Acknowledged

Note

JetBit acknowledged this finding. The keeper automation only runs during the presale, while the JET/USDT pool is seeded and held at the $1.00 reference, where the USDT amount and the liquidity delta match. JetBit retires the automation once trading starts and sizes any later add manually to the live price. A failed add reverts atomically with no loss of funds.

[M-02] JetPresale#buy - No minimum-output slippage bound on purchase

Description

`JetPresale.buy` takes only a `usdtAmount` budget and computes the JET delivered from the live `sold` state at execution time, with no caller-supplied minimum output. Because the ladder price rises as tranches fill, other purchases confirming between a user's on-chain quote and their `buy` transaction can advance the active tranche, so the same USDT buys fewer JET than the interface displayed while the call still succeeds. The buyer has no way to say "revert if I receive less than X JET." Source identifier: `TP-FC-B9`.

Vulnerability Details

`buy` accepts a single parameter and derives the output from `_quote` at call time (`src/sale/JetPresale.sol:133-155`):

```
function buy(uint256 usdtAmount) external nonReentrant returns (uint256 jetOut) {
    if (usdtAmount == 0) revert ZeroAmount();
    if (!isActive()) revert SaleNotActive();

    uint256 usdtUsed;
    (jetOut, usdtUsed) = _quote(usdtAmount);
    if (jetOut == 0 || usdtUsed == 0) revert SoldOut();
    ...
    jetToken.safeTransfer(msg.sender, jetOut);
    emit Purchased(msg.sender, usdtUsed, jetOut, toLp, toFunds);
}
```

`_quote` reads `sold` and walks the price tranches, so the price applied is whatever the ladder cursor is at the moment of execution (`src/sale/JetPresale.sol:168-189`):

```
uint256 soldSoFar = sold;
...
while (remaining > 0 && soldSoFar < cap) {
    uint256 idx = soldSoFar / trancheSize;
    if (idx >= numTranches) break;
    uint256 price = _priceAt(idx);
    uint256 trancheRemainingJet = ((idx + 1) * trancheSize) - soldSoFar;
```

```
    uint256 jetAtPrice = (remaining * WAD) / price;
```

The ladder itself is immutable once a sale has begun: `setLadder` reverts when `sold != 0` (`src/sale/JetPresale.sol:263-273`). That prevents an administrator from repricing mid-sale, but it does not bind an individual purchase to a minimum result — the cursor `sold` still advances with every buy, so `price` is higher for the later transaction. The frontend exposes the same one-parameter ABI (`tj-frontend-main/constant/presale.ts:55`), confirming callers cannot pass a slippage bound.

Existing controls: the ladder is frozen after the first sale and payment is bounded by the offered USDT budget (`usdtUsed <= usdtAmount`); no arbitrary post-start administrator repricing was proved. Limits: the presale is not one of the two user-supplied Pool/Vault addresses, so its current implementation and live sale state remain unverified; classify this as missing purchase-intent protection, not permissionless access to another party's funds.

Proof of Concept

The test drives two buyers through the real `buy(uint256)` / `quote(uint256)` signatures. Buyer A reads a quote at the opening tranche; buyer B fills the rest of that tranche first; buyer A's `buy` then executes one tranche higher and receives fewer JET than quoted.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "forge-std/Test.sol";
import { JetPresale, JetPresaleInitParams } from "src/sale/JetPresale.sol";
import { ERC1967Proxy } from
"@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import { MockERC20 } from "./mocks/MockERC20.sol"; // 18dp ERC20 with mint()

contract PresaleSlippageTest is Test {
    JetPresale sale;
    MockERC20 usdt;
    MockERC20 jet;
```

```
    address owner = address(0xA11CE);
    address alice = address(0xA);
    address bob   = address(0xB);

    function setUp() public {
        usdt = new MockERC20("USDT", "USDT", 18);
        jet  = new MockERC20("JET", "JET", 18);
        JetPresale impl = new JetPresale();
        JetPresaleInitParams memory p = JetPresaleInitParams({
            owner: owner, usdt: address(usdt), jetToken: address(jet),
            liquidityWallet: address(0x1), fundsWallet: address(0x2),
            openPrice: 0.65e18, stepBps: 1000, trancheSize: 50_000_000e18,
            numTranches: 5, lpSplitBps: 1000, saleStart: 0, saleEnd: 0
        });
        sale = JetPresale(address(new ERC1967Proxy(address(impl),
abi.encodeCall(JetPresale.initialize, (p)))));
        jet.mint(address(sale), 250_000_000e18); // fund deliveries
        // put both buyers one JET-cent short of the tranche boundary
        usdt.mint(alice, 40_000_000e18); usdt.mint(bob, 40_000_000e18);
        vm.prank(alice); usdt.approve(address(sale), type(uint256).max);
        vm.prank(bob);   usdt.approve(address(sale), type(uint256).max);
    }

    function test_purchaseCrossesTrancheAndUnderDelivers() public {
        // Alice quotes at tranche 0 ($0.65): 1000e18 USDT -> ~1538.46e18 JET
        (uint256 quotedJet,) = sale.quote(1_000e18);

        // Bob buys enough to exhaust tranche 0 before Alice's tx lands.
        vm.prank(bob); sale.buy(32_500_000e18); // ~50M JET at $0.65 fills tranche 0

        // Alice's buy now executes at tranche 1 ($0.715): fewer JET for the same
USDT.
        vm.prank(alice); uint256 got = sale.buy(1_000e18);

        assertLt(got, quotedJet); // delivered less than the displayed quote, tx
still succeeded
    }
}
```

Impact

A buyer can receive materially fewer JET than the amount the interface quoted for the same USDT, whenever concurrent purchases push the ladder into a higher tranche between quote and execution, and the transaction still succeeds because no minimum-output check can fail. The loss per buyer is bounded by the ladder step (default +10% per tranche crossed) and is a purchase-intent gap, not permissionless access to another party's funds. The presale's deployed implementation and live sale state were not verified, so no realized monetary loss is asserted, only the missing slippage protection in the code.

Recommendation

Add a caller-supplied minimum-output parameter to `buy` and revert when the execution-time quote falls below it.

```
-    function buy(uint256 usdtAmount) external nonReentrant returns (uint256 jetOut)
{
+    function buy(uint256 usdtAmount, uint256 minJetOut) external nonReentrant
returns (uint256 jetOut) {
         if (usdtAmount == 0) revert ZeroAmount();
         if (!isActive()) revert SaleNotActive();

         uint256 usdtUsed;
         (jetOut, usdtUsed) = _quote(usdtAmount);
         if (jetOut == 0 || usdtUsed == 0) revert SoldOut();
+        if (jetOut < minJetOut) revert SlippageExceeded(jetOut, minJetOut);
```

Keeping the old single-argument selector as a `minJetOut = 0` wrapper preserves compatibility for callers that opt out.

Status

Acknowledged

Note

JetBit acknowledged this finding. The price only changes when a purchase crosses a tranche boundary, and JetBit does not expect the sale to move past the first tranche.

JetPresale is upgradeable, and JetBit will add a minimum output parameter if demand approaches a boundary.

[M-03] Pool#onStake - 256-bit accumulator debt narrowed into 128-bit position fields

Description

`Pool.onStake` computes a new position's `debtProfit` and `debtLoss` as full 256-bit products of the WAD-scaled monotonic accumulators, but `LPStaking._mintStakePosition` stores them in the `ILPPositionNFT.Position` struct, whose debt fields are `uint128`, through the checked `_toU128` helper. The accumulators `accProfitPerShare` and `accLossPerShare` only ever grow and are never reset when total shares fall to zero, and permissionless `injectProfit` (plus role-gated loss draining) can inflate profit-per-share sharply while shares are small. Once an accumulator is large enough that `shares * acc / WAD` exceeds `2^128 - 1`, `_toU128` reverts, so every new stake above the representable-debt limit is persistently rejected. This is a staking-availability defect for new positions; it does not establish theft, corrupted debt, or failure to withdraw existing positions. Source identifier: `TP-TG-F07-ACCUMULATOR-DEBT-WIDTH`.

Vulnerability Details

`onStake` derives the debts as `uint256` from the accumulators (`src/lp/Pool.sol:97-103`):

```
shares = netAmount;

totalShares += shares;
totalAtRiskShares += atRiskSharesIn;

debtProfit = shares.mulDivDown(accProfitPerShare, WAD);
debtLoss = shares.mulDivDown(accLossPerShare, WAD);
```

`accProfitPerShare` and `accLossPerShare` are `uint256` storage that only increase (`src/lp/Pool.sol:48-53`). `_accrueProfit` adds `amount.mulDivDown(WAD, totalShares)` (`src/lp/Pool.sol:216`), so a fixed `amount` injected while `totalShares` is tiny produces a very large per-share increment. `injectProfit` is open to any caller and only requires `totalShares != 0` (`src/lp/Pool.sol:164-170`):

```
function injectProfit(uint256 amount) external {
```

```
    if (amount == 0) revert ZeroAmount();
    if (totalShares == 0) revert NoSharesToDistribute();
    _usdt.safeTransferFrom(msg.sender, address(this), amount);
    _accrueProfit(amount);
    emit ProfitInjected(msg.sender, amount, accProfitPerShare);
}
```

`drainToVault` similarly grows `accLossPerShare` (`src/lp/Pool.sol:203`). Nothing resets either accumulator when shares return to zero, so the WAD-scaled value carries forward across the pool's whole life. The `uint256` debts then meet the `uint128` position fields in `LPStaking._mintStakePosition` (`src/lp/LPStaking.sol:363-372`):

```
(uint256 shares, uint256 debtProfit, uint256 debtLoss) =
    IPool(_requireRegistryAddr(KEY_POOL)).onStake(c.net, c.atRisk, c.reserved);

ILPPositionNFT.Position memory p = ILPPositionNFT.Position({
    shares: _toU128(shares),
    ...
    debtProfit: _toU128(debtProfit),
    debtLoss: _toU128(debtLoss),
```

`_toU128` reverts on overflow rather than truncating (`src/lp/LPStaking.sol:449-452`):

```
function _toU128(uint256 x) internal pure returns (uint128) {
    require(x <= type(uint128).max, "u128 overflow");
    return uint128(x);
}
```

The `Position` struct's debt fields are `uint128` (`src/interfaces/ILPPositionNFT.sol:5-13`). So a stake whose computed `debtProfit` or `debtLoss` exceeds `2^128 - 1` reverts at mint — the checked conversion rejects the new position rather than silently corrupting its debt.

Existing controls: positive amount and nonzero shares are required; funding is by transfer; loss is role-gated; the checked conversion prevents storing corrupted debt; UUPS upgrade authority remains. Supported impact: low-share states can drive persistent rejection of new stakes above the representable-debt limit, and the code provides no accumulator rebasing or reset. Limits: existing-position withdrawal failure,

direct theft, and realized monetary loss are not established. Actual LPStaking/source parity, current debt/accumulator/total-share state, funding, and permissionless low-share reachability in a live deployment were not verified; the external validation report's six cited test files are absent from this workspace and no execution transcript was supplied, so its claimed passing tests, numerical outcomes, and attack costs remain attributed but independently unverified.

Proof of Concept

Numeric walkthrough of the overflow threshold, then a snippet exercising the real path. Seed a small share base, inject profit so the per-share accumulator is huge, then have any user attempt a normal-sized stake.

```
WAD                = 1e18
totalShares        = 1e18            (one small position, ~1 share-unit)
injectProfit(amt)  amt = 1e20 USDT
  increment        = amt * WAD / totalShares
                   = 1e20 * 1e18 / 1e18 = 1e20
  accProfitPerShare = 1e20   (already > previous, monotonic)

New staker deposits netAmount = 2e18 (shares = 2e18):
  debtProfit = shares * accProfitPerShare / WAD
             = 2e18 * 1e20 / 1e18 = 2e20        // still fits u128 here
Repeat injections while shares stay small so accProfitPerShare climbs
past ~1.7e38; then for any new stake:
  debtProfit = shares * acc / WAD  > 2^128-1 (~3.4e38)
  _toU128(debtProfit) -> require(x <= type(uint128).max) FAILS -> revert "u128
overflow"
```

```
// Real signatures exercised:
pool.injectProfit(1e20);          // src/lp/Pool.sol:164 — permissionless, grows
accProfitPerShare
// ... repeated while totalShares is small ...
// then any user stake routes through:
IPool(pool).onStake(netAmount, atRisk, reserved);   // src/lp/Pool.sol:88 -> returns
uint256 debtProfit
// LPStaking then narrows it:
```

```
p.debtProfit = _toU128(debtProfit); // src/lp/LPStaking.sol:449 -> reverts once
debtProfit > 2^128-1
```

Every subsequent stake whose `shares * accProfitPerShare / WAD` exceeds `2^128 - 1` reverts at `_toU128`, blocking new-position creation until the accumulator is rebased, which the code cannot do.

Impact

New staking can be persistently blocked: once the never-reset profit or loss accumulator grows large enough (reachable via permissionless `injectProfit` while total shares are small), any stake whose narrowed `debtProfit`/`debtLoss` exceeds `2^128 - 1` reverts at `_toU128`, and the contract has no accumulator reset or rebasing procedure to recover. Scope is new-stake availability only; existing-position withdrawal failure, direct theft, and realized monetary loss are not established, and the checked conversion rejects rather than silently corrupts. Live LPStaking/source parity, current accumulator and share state, and low-share reachability in a deployment were not verified.

Recommendation

Keep accumulator snapshots representable across the supported stake range: reset (or checkpoint-rebase) the per-share accumulators on empty-pool transitions, and reject or scale injections that would push the per-share value past a bound the `uint128` debt fields can hold.

```
function onUnstake(...) external returns (uint256 poolPayout) {
         _onlyLPStaking();
         poolPayout = _computePoolPayout(shares_, atRiskShares_, debtProfit_,
debtLoss_);
         totalShares -= shares_;
         totalAtRiskShares -= atRiskShares_;
+        // When the pool empties, rebase accumulators so a new cohort does not
+        // inherit an unboundedly large per-share snapshot that overflows u128 debt.
+        if (totalShares == 0) {
+            accProfitPerShare = 0;
+            accLossPerShare = 0;
+        }
```

```
    }
```

```
function _accrueProfit(uint256 amount) internal {
        uint256 increment = amount.mulDivDown(WAD, totalShares);
+       // Bound the per-share accumulator so shares*acc/WAD stays within u128.
+       require(accProfitPerShare + increment <= MAX_ACC_PER_SHARE, "acc overflow");
        accProfitPerShare += increment;
    }
```

Status

Resolved

Note

JetBit reset both per-share accumulators to zero whenever the Pool empties (commit 45c4ac8). In the fix review commit, the Pool moved to unit accounting, and new positions store Pool units instead of accumulator debt, so the uint128 overflow path no longer applies to new stakes.

[M-04] LPStaking#reinitialize2 - Migration hooks version-gated but not caller-authenticated

Description

`LPStaking.reinitialize2` and `reinitialize3` are guarded only by OpenZeppelin's `reinitializer(N)` version check, which prevents repeat execution but does not authenticate the caller. On a proxy that has not yet consumed the corresponding version, any account can invoke these tier-writing migration functions directly, rather than only the intended `upgradeToAndCall` migration. `reinitialize2` overwrites the lock-tier defaults, which would revert any privileged tuning applied to future positions, and `reinitialize3` consumes version 3 and thereby blocks `reinitialize2` from running afterward, disrupting the intended migration sequence. Existing minted positions' `lockEnd` is not rewritten. Source identifier: `TP-TG-F14-PERMISSIONLESS-MIGRATIONS`.

Vulnerability Details

Both hooks carry only the version modifier, with no role or ownership check (`src/lp/LPStaking.sol:120-133`):

```
function reinitialize2() external reinitializer(2) {
    _setDefaultTiers();
}
...
function reinitialize3() external reinitializer(3) {
    _lockDurations[ADMIN_LOCK_TIER] = DEFAULT_ADMIN_LOCK_DURATION;
    _insuranceBpsValues[ADMIN_LOCK_TIER] = 0;
}
```

By contrast `initialize` uses the `initializer` modifier and pulls trusted wiring from the registry (`src/lp/LPStaking.sol:108-114`). `reinitialize2` calls `_setDefaultTiers`, which unconditionally writes the production lock durations and insurance values (`src/lp/LPStaking.sol:135-145`), so calling it after an admin has tuned those values silently restores the defaults for all future positions. `reinitializer(N)` from the pinned OpenZeppelin `Initializable` only checks and advances the stored initialized version; it does not consult access control. The intended path is an atomic upgrade, `upgradeToAndCall(newImpl, abi.encodeCall(LPStaking.reinitialize3, ()))`

(`script/UpgradeLPStaking.s.sol:44-46`), but nothing forces the migration to run inside that upgrade. If a proxy's initialized version still sits below the target when a new implementation exposing these functions is live, an unprivileged caller can call `reinitialize3` first, consuming version 3 and permanently preventing `reinitialize2` (version 2) from being applied, or call `reinitialize2` to wipe tuned tiers. The `standard-input.json` compile artifact (`audit-reverification/chain/compile-comparison/standard-input.json:1`) documents the compiled sources under review.

Existing controls: implementation constructors call `_disableInitializers()`; version gating prevents repeat calls; upgrades can atomically call the migration; the current `initialize` sets defaults. Supported impact: on an eligible proxy, an unprivileged migration call can alter future tier configuration or disrupt the intended migration sequence. Limits: current LPStaking implementation parity and the proxy's initialized version, and the actual upgrade/migration rollout, were not verified.

Proof of Concept

The test upgrades a proxy to an implementation exposing the reinitializers without wiring the migration into `upgradeToAndCall`, then a non-owner front-runs the migration.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "forge-std/Test.sol";
import { LPStaking } from "src/lp/LPStaking.sol";

interface IUUPS { function upgradeToAndCall(address, bytes calldata) external
payable; }

contract PermissionlessMigrationTest is Test {
    address constant PROXY = address(0xBEEF);      // live LPStaking proxy at version
1
    address owner    = address(0xA11CE);
    address attacker = address(0xBAD);
```

```
    function test_unprivilegedCallerConsumesReinitVersion() public {
        // Owner upgrades to a new impl but forgets to bundle the migration call.
        LPStaking newImpl = new LPStaking();
        vm.prank(owner);
        IUUPS(PROXY).upgradeToAndCall(address(newImpl), ""); // no reinitialize in
the same tx

        // reinitialize3 is only reinitializer(3); no auth. Attacker consumes v3
first.
        vm.prank(attacker);
        LPStaking(PROXY).reinitialize3(); // succeeds — advances initialized version
to 3

        // Now the intended reinitialize2 (version 2) can never run: version already
> 2.
        vm.prank(owner);
        vm.expectRevert(); // Initializable: invalid initialization
        LPStaking(PROXY).reinitialize2();
    }
}
```

Impact

On any proxy whose initialized version is still below the migration target when an implementation exposing `reinitialize2`/`reinitialize3` is live, an unprivileged account can wipe tuned lock-tier configuration back to defaults (affecting future positions) or consume version 3 to permanently block the version-2 migration, disrupting the operator's intended sequence. Existing minted positions' `lockEnd` is not rewritten, so already-locked stakes are unaffected. Current LPStaking implementation parity, the proxy's actual initialized version, and the real upgrade rollout were not verified, so this is a code-level authorization gap rather than a demonstrated live exploit.

Recommendation

Authenticate the migration hooks with the same authority that gates upgrades, and run each migration atomically with its upgrade so no window exists for an outside caller.

```
-    function reinitialize2() external reinitializer(2) {
+    function reinitialize2() external reinitializer(2) onlyRole(UPGRADER_ROLE) {
         _setDefaultTiers();
     }
     ...
-    function reinitialize3() external reinitializer(3) {
+    function reinitialize3() external reinitializer(3) onlyRole(UPGRADER_ROLE) {
         _lockDurations[ADMIN_LOCK_TIER] = DEFAULT_ADMIN_LOCK_DURATION;
         _insuranceBpsValues[ADMIN_LOCK_TIER] = 0;
     }
```

Reuse whatever authority `_authorizeUpgrade` enforces so the check cannot drift from the upgrade path.

Status

Acknowledged

Note

JetBit acknowledged this finding. Every LPStaking upgrade runs as a single transaction that bundles the migration call through upgradeToAndCall, so the new implementation and its reinitializer go live together. The hooks only change lock tier settings for future positions and cannot move funds or rewrite existing positions.

[M-05] JetPresale#_quote - Quote token raw units assumed to be 18 decimals

Description

`JetPresale._quote` treats the raw `usdtAmount` as a WAD-scaled (18-decimal) figure when dividing by the WAD-denominated ladder price, and never reads or normalizes the quote token's actual decimals. The ladder price `openPrice` is USDT-per-JET in WAD and JET is 18 decimals, so the arithmetic `remaining * WAD / price` only yields the correct JET amount when the quote token is also 18 decimals. Wired to a 6-decimal quote token, every purchase is under-scaled by roughly `1e12`, so a buyer paying a nominal amount receives vastly too little JET. This is a source-level unit-compatibility defect for a non-18-decimal wiring; the current deploy script defaults to 18-decimal BSC USDT, so no matched deployment was found. Source identifier: `TP-TG-F13-DECIMALS`.

Vulnerability Details

`WAD` is fixed at `1e18` (`src/sale/SaleShared.sol:14`), and `openPrice` is documented as WAD USDT-per-JET (`src/sale/JetPresale.sol:67`). `_quote` converts a USDT budget to JET by multiplying by `WAD` and dividing by the WAD price (`src/sale/JetPresale.sol:168-189`):

```
uint256 price = _priceAt(idx);
uint256 trancheRemainingJet = ((idx + 1) * trancheSize) - soldSoFar;

uint256 jetAtPrice = (remaining * WAD) / price;
```

Here `remaining` is the raw quote-token budget as passed into `buy` (`src/sale/JetPresale.sol:133`). The dimensional identity the formula assumes is `jet(18dp) = usdt(18dp) * 1e18 / price(18dp)`. If the quote token carries 6 decimals, `remaining` is smaller by `1e12` for the same nominal dollar value, so `jetAtPrice` comes out `1e12` times too small. There is no `IERC20Metadata(usdt).decimals()` read anywhere in the sale path and no normalization constant; the code implicitly assumes an 18-decimal quote token. The deploy script defaults the quote token to BSC USDT, which is 18 decimals (`script/DeployPresale.s.sol:30`), so the shipped default happens to be safe, but the owner may pass any `USDT_ADDRESS`, and USDT on most other chains (Ethereum, Tron, many L2s) is 6 decimals.

Existing controls: the current presale deploy script defaults to BSC USDT (18dp); the owner can choose other initialization parameters; overflow, tiny/zero-output, and final-transfer reverts remain atomic. Supported impact: with a WAD-denominated price and 18-decimal JET, `_quote` treats quote raw units as 18-decimal units, so a 6-decimal quote token is under-scaled and buyers receive far too little JET for their nominal payment. Limits: no presale deployment matched this wiring; the ~1e12 factor describes the specified 6-versus-18-decimal setup, not every chain or ladder state.

Proof of Concept

Numeric walkthrough with a 6-decimal quote token at the default opening price, followed by the exact expression that under-scales.

```
openPrice (WAD)     = 0.65e18            // $0.65 per JET
Buyer pays          = 1,000 USDT
  18dp token: remaining = 1_000e18
  6dp  token: remaining = 1_000e6

jetAtPrice = remaining * WAD / price     // src/sale/JetPresale.sol:180

18dp: 1_000e18 * 1e18 / 0.65e18 = 1_538.46e18 JET   (correct: ~1538 JET)
 6dp: 1_000e6  * 1e18 / 0.65e18 = 1_538.46e6  JET   = 0.00000000153846 JET
```

```
// Real path — buy() forwards the raw budget straight into _quote:
function buy(uint256 usdtAmount) external nonReentrant returns (uint256 jetOut) {
// :133
    ...
    (jetOut, usdtUsed) = _quote(usdtAmount);   // usdtAmount is RAW token units
}
// _quote assumes 18dp:
uint256 jetAtPrice = (remaining * WAD) / price; // :180 — no decimals normalization
```

For a 6-decimal quote token the buyer paying $1,000 receives `1_538.46e6` base units of an 18-decimal JET, i.e. about `1.5e-9` JET, a `1e12` shortfall versus the intended ~1,538 JET, while `usdtUsed` (and thus the USDT actually pulled) is also `1e12` smaller than a dollar-equivalent purchase.

Impact

If the presale is initialized with a 6-decimal quote token (USDT on Ethereum, Tron, and most non-BSC chains), `_quote` under-scales the JET output by roughly `1e12`, so buyers receive almost no JET for a nominal dollar payment; the mirror effect is that `usdtUsed` is likewise `1e12` too small, so the ladder and raised total are meaningless on that wiring. The shipped deploy default is 18-decimal BSC USDT, so no matched live deployment was found and the `1e12` figure is specific to the 6-versus-18-decimal case, not every chain or ladder state. This is a source compatibility defect that manifests only under a non-18-decimal quote-token wiring the owner could choose.

Recommendation

Read the quote token's decimals at initialization and normalize the budget to WAD before the ladder math, or reject any quote token that is not 18 decimals during initialization.

```
+    uint256 private _quoteScale; // 1e(18 - quoteDecimals), set at init

     function initialize(JetPresaleInitParams calldata p) external initializer {
         ...
         usdt = IERC20(p.usdt);
+        uint8 d = IERC20Metadata(p.usdt).decimals();
+        require(d <= 18, "quote decimals");
+        _quoteScale = 10 ** (18 - d); // 1 for 18dp, 1e12 for 6dp
```

```
function _quote(uint256 usdtAmount) internal view returns (uint256 jetOut, uint256
usdtUsed) {
-        uint256 remaining = usdtAmount;
+        uint256 remaining = usdtAmount * _quoteScale; // normalize to WAD before
ladder math
         ...
+        // divide usdtUsed back down by _quoteScale before charging the buyer,
+        // so token transfers stay in the quote token's native units.
     }
```

Status

Acknowledged

Note

JetBit acknowledged this finding. According to JetBit, the presale is deployed with BSC USDT, which uses 18 decimals, so the quote math is correct for the live configuration and the issue does not trigger.

Low

[L-01] JetLiquidityLocker#initializePool - No adoption path for an already-initialized external pool

Description

`JetLiquidityLocker.initializePool` builds the external PancakeSwap Infinity `PoolKey` and calls the CL pool manager's `initialize` for the JET/USDT pair. Deployment of the locker and this owner-only setup step are separate operations, spread across the deploy scripts. If a pool for the chosen `(currency0, currency1, fee, tickSpacing, hooks)` key already exists on the target chain, the upstream `initialize` reverts, and the locker offers no way to adopt that pre-existing pool. Ownership of the locker stays protected throughout, so this is a deployment-correctness gap, not a capture. Source identifier: `TP-FC-B8`.

Vulnerability Details

`initializePool` is `onlyOwner`, guards against re-initialization with `poolInitialized`, then constructs the external pool key and initializes it on the shared PancakeSwap CL pool manager:

```
function initializePool(uint24 fee, int24 tickSpacing) external onlyOwner {
    if (poolInitialized) revert PoolAlreadyInitialized();
    require(tickSpacing > 0, "tickSpacing");

    // parameters: hook bits [0..16) = 0 (no hooks); tickSpacing (int24) at [16..40).
    bytes32 parameters = bytes32(uint256(uint24(tickSpacing)) << 16);

    PoolKey memory k = PoolKey({
        currency0: currency0,
        currency1: currency1,
        hooks: IHooks(address(0)),
        poolManager: IPoolManager(address(CL_POOL_MANAGER)),
        fee: fee,
        parameters: parameters
    });
```

```
    poolKey = k;
    ...
    CL_POOL_MANAGER.initialize(k, SQRT_PRICE_ONE);
    poolInitialized = true;
    emit PoolInitialized(fee, tickSpacing, tickLower, tickUpper);
}
```

The pool manager is a shared, external singleton (`CL_POOL_MANAGER`), and its `initialize` rejects a pool key that has already been initialized. The JET/USDT pair, a zero-hooks key, and a common fee/spacing tier are exactly the kind of key another party could have created first. When that happens, `CL_POOL_MANAGER.initialize(k, SQRT_PRICE_ONE)` reverts, the whole `initializePool` transaction rolls back, and `poolKey`/`poolInitialized`/`tickLower`/`tickUpper` are never persisted. There is no alternate entrypoint that accepts an already-initialized external pool and records its key so the locker can operate against it.

Because `poolInitialized` never flips to `true`, `addLiquidity` stays blocked:

```
function addLiquidity(uint256 usdtAmount) external nonReentrant returns (uint128
liquidity) {
    if (msg.sender != keeper && msg.sender != owner()) revert NotKeeper();
    if (!poolInitialized) revert PoolNotInitialized();
    ...
}
```

The two deploy scripts confirm the split lifecycle. `DeployPresale.s.sol` deploys the locker but explicitly defers pool creation to a later owner action ("`locker.initializePool(fee, tickSpacing)` — on mainnet, once (needs live Pancake)"). `RedeployLocker.s.sol` broadcasts a fresh locker and then calls `initializePool` as a distinct step in the same script. Either flow reverts if the matching external pool already exists.

Existing controls: `initializePool` is `onlyOwner`, so this is not unauthorized owner-state initialization. A failed manager call rolls back the local key and flag, leaving no half-written state. The owner can retry with a different valid fee/tickSpacing key, so permanent capture is not established.

Limits: This concerns `JetLiquidityLocker` and its external PancakeSwap pool, not the JetBit custodial `Pool` proxy. An already-initialized locker is past this deployment window. The current locker instance and setup state, and the exact deployed-code parity of the external CL pool manager, are unresolved and were not verified against a live deployment.

Proof of Concept

A minimal Foundry test with a mock CL pool manager that mimics the upstream rule (revert on a re-initialized key) shows `initializePool` reverting for the intended fee/spacing and leaving the locker unusable:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import { Test } from "forge-std/Test.sol";
import { JetLiquidityLocker } from "src/lp/JetLiquidityLocker.sol";

// Mimics PancakeSwap CLPoolManager.initialize: a given pool key can be
// initialized exactly once; a second call for the same key reverts.
contract MockCLPoolManager {
    mapping(bytes32 => bool) public initialized;

    function initialize(bytes calldata keyEncoded, uint160) external {
        bytes32 id = keccak256(keyEncoded);
        require(!initialized[id], "pool already initialized");
        initialized[id] = true;
    }
}

contract LockerAdoptionTest is Test {
    JetLiquidityLocker locker;

    function test_initializePool_reverts_when_external_pool_exists() public {
        // Deploy the locker (owner = this test) with JET/USDT/keeper addresses.
        locker = new JetLiquidityLocker(address(this), address(0xJET),
address(0xUSDT), address(this));
```

```
        // Someone initialized the JET/USDT pool for the chosen tier first.
        // The upstream manager now rejects the locker's own initialize.
        vm.expectRevert(); // upstream "pool already initialized"
        locker.initializePool(15000, 50);

        // No adoption path exists, so the locker is stuck: addLiquidity is gated.
        vm.expectRevert(); // PoolNotInitialized()
        locker.addLiquidity(0);
    }
}
```

The `MockCLPoolManager` stands in for the shared external manager because the real singleton is not deployed in the unit test; the assertion of interest is that `initializePool` has no branch that records a pre-existing pool key.

Impact

If the JET/USDT pool for the selected fee/tickSpacing already exists when the owner runs setup, `initializePool` reverts and the locker cannot be brought online for that key: `addLiquidity` stays blocked by the `poolInitialized` gate. The owner can work around it by choosing a different valid fee/spacing tier, so this does not establish a permanent brick and no realized monetary loss is demonstrated. Preconditions are a matching external pool created ahead of the owner, and the whole issue applies only within the one-time deployment window; an already-initialized locker is unaffected. Current deployed parity of the external pool manager is unresolved.

Recommendation

Add an owner-only adoption path that records an existing external pool key without calling `initialize`, so a pre-created matching pool does not brick setup. Compute or accept the same key, verify the pool exists upstream, then persist state.

```
--- a/src/lp/JetLiquidityLocker.sol
+++ b/src/lp/JetLiquidityLocker.sol
@@
     function initializePool(uint24 fee, int24 tickSpacing) external onlyOwner {
         if (poolInitialized) revert PoolAlreadyInitialized();
```

```
         require(tickSpacing > 0, "tickSpacing");
@@
         CL_POOL_MANAGER.initialize(k, SQRT_PRICE_ONE);
         poolInitialized = true;
         emit PoolInitialized(fee, tickSpacing, tickLower, tickUpper);
     }
+
+    /// @notice Adopt an already-initialized external JET/USDT pool for the given
+    ///         fee/tickSpacing instead of creating a new one. Owner-only, one-time.
+    function adoptPool(uint24 fee, int24 tickSpacing) external onlyOwner {
+        if (poolInitialized) revert PoolAlreadyInitialized();
+        require(tickSpacing > 0, "tickSpacing");
+        bytes32 parameters = bytes32(uint256(uint24(tickSpacing)) << 16);
+        PoolKey memory k = PoolKey({
+            currency0: currency0,
+            currency1: currency1,
+            hooks: IHooks(address(0)),
+            poolManager: IPoolManager(address(CL_POOL_MANAGER)),
+            fee: fee,
+            parameters: parameters
+        });
+        // Revert if the pool is NOT already initialized upstream (sqrtPriceX96 ==
0).
+        // Uses the manager's pool-state getter; adjust to the deployed manager ABI.
+        require(_externalPoolExists(k), "pool not initialized");
+        poolKey = k;
+        tickLower = (MIN_TICK / tickSpacing) * tickSpacing;
+        tickUpper = (MAX_TICK / tickSpacing) * tickSpacing;
+        poolInitialized = true;
+        emit PoolInitialized(fee, tickSpacing, tickLower, tickUpper);
+    }
```

Status

Acknowledged

Note

JetBit acknowledged this finding. The issue only affects the one-time initialization window, and the locker's pool is already initialized, so that window has passed. No funds are at risk.

[L-02] LPPositionNFT#tokenURI - Returns a raw JSON object string instead of a metadata URI

Description

`LPPositionNFT.tokenURI` returns a bare, unencoded JSON object string rather than a URI. The ERC-721 metadata extension specifies that `tokenURI` return "A distinct Uniform Resource Identifier (URI) for a given asset", typically an `https://` link or a `data:application/json; base64,...` data URI that resolves to the metadata document. Returning the JSON body directly means URI-based clients (wallets, marketplaces) cannot resolve or render the metadata through the standard path. The team chose plain JSON intentionally and expects the frontend to read position data directly, so this is an interoperability-correctness issue, not a fund-security defect. Source identifier: `TP-TG-L17-METADATA-URI`.

Vulnerability Details

`tokenURI` builds and returns a JSON object as a plain string:

```
function tokenURI(uint256 tokenId) public view override returns (string memory) {
    if (!_exists(tokenId)) revert PositionNotFound(tokenId);
    Position memory p = _positions[tokenId];
    // Minimal on-chain JSON. Intentionally plain (no base64, no image).
    // Frontends can render richer metadata from the same data.
    return string.concat(
        '{"tokenId":"',
        Strings.toString(tokenId),
        '","shares":"',
        Strings.toString(p.shares),
        '","atRiskShares":"',
        Strings.toString(p.atRiskShares),
        '","reservedAmount":"',
        Strings.toString(p.reservedAmount),
        '","lockEnd":"',
        Strings.toString(p.lockEnd),
        '","lockTier":"',
        Strings.toString(p.lockTier),
        '"}'
```

```
    );
}
```

The return value begins with `{` and is a JSON document, not a URI. The ERC-721 `IERC721Metadata` interface documents `tokenURI` as returning a URI; conforming consumers fetch that URI and parse the resource at the other end. A common on-chain pattern is to wrap the same JSON as a data URI, for example prefixing `data:application/json; base64,` followed by the base64 of the JSON, or `data:application/json;utf8,` with the raw JSON. The current implementation emits neither prefix, so a client that treats the return value as a URI has nothing valid to dereference.

The contract itself is otherwise well-formed for its purpose: `tokenURI` reverts with `PositionNotFound(tokenId)` for a nonexistent token, and the numeric fields are serialized as decimal strings via `Strings.toString`, which is unambiguous for a custom reader. The finding is strictly about the URI contract of the standard, not the field contents.

Existing controls: the function performs an NFT existence check before reading state; the fields are structured numeric strings; and a custom UI can read the same position data directly, which is the intended consumption path per the code comment.

Limits: actual wallet and marketplace behavior against this return value, and the deployed NFT's on-chain parity, were not exercised on a live client; the conclusion follows from the standard's URI wording versus the returned JSON body.

Proof of Concept

This is an interface-conformance issue; the walkthrough contrasts the standard's expectation with the returned value.

The ERC-721 metadata interface is:

```
interface IERC721Metadata {
    function tokenURI(uint256 _tokenId) external view returns (string memory);
```

```
    // returns "A distinct Uniform Resource Identifier (URI) for a given asset."
}
```

A standard consumer treats the result as a URI and dereferences it. For token id `1` with `shares = 100`, `atRiskShares = 0`, `reservedAmount = 0`, `lockEnd = 0`, `lockTier = 0`, the current `tokenURI` returns:

```
{"tokenId":"1","shares":"100","atRiskShares":"0","reservedAmount":"0","lockEnd":"0","
lockTier":"0"}
```

A URI parser given that string has no scheme (`https:`, `ipfs:`, `data:`), so there is nothing to fetch or decode; a strict client shows no metadata. A conforming value for the same data would instead be a data URI whose body decodes to that JSON:

```
// Standards-conforming shape: a data: URI, not the raw JSON object.
string memory uri = string.concat(
    "data:application/json; base64,",
    Base64.encode(bytes(jsonBody)) // jsonBody = the object above
);
// uri now starts with "data:application/json; base64," -> resolvable by URI clients.
```

Impact

URI-based metadata clients (wallets and marketplaces that call `tokenURI` and expect a dereferenceable URI) may fail to resolve or display the position's metadata, degrading interoperability of the `LPPositionNFT`. There is no loss of funds and no effect on position accounting: transfers, lock state, and shares are unaffected because they do not depend on `tokenURI`. The consequence is limited to display/discoverability on third-party clients, and only matters where such a client is used; the project's own frontend reads the data directly. Actual client behavior and deployed NFT parity remain unverified.

Recommendation

Wrap the existing JSON as a `data:application/json` URI so the return value is a resolvable URI while keeping the same on-chain data. Base64 encoding is the most broadly compatible form.

```
--- a/src/lp/LPPositionNFT.sol
+++ b/src/lp/LPPositionNFT.sol
@@
 import { Strings } from "@openzeppelin/contracts/utils/Strings.sol";
+import { Base64 } from "@openzeppelin/contracts/utils/Base64.sol";
@@
     function tokenURI(uint256 tokenId) public view override returns (string memory)
{
         if (!_exists(tokenId)) revert PositionNotFound(tokenId);
         Position memory p = _positions[tokenId];
-        // Minimal on-chain JSON. Intentionally plain (no base64, no image).
-        // Frontends can render richer metadata from the same data.
-        return string.concat(
+        string memory json = string.concat(
             '{"tokenId":"',
             Strings.toString(tokenId),
             '","shares":"',
             Strings.toString(p.shares),
             '","atRiskShares":"',
             Strings.toString(p.atRiskShares),
             '","reservedAmount":"',
             Strings.toString(p.reservedAmount),
             '","lockEnd":"',
             Strings.toString(p.lockEnd),
             '","lockTier":"',
             Strings.toString(p.lockTier),
             '"}'
         );
+        return string.concat(
+            "data:application/json; base64,",
+            Base64.encode(bytes(json))
+        );
     }
```

Status

Acknowledged

Note

JetBit acknowledged this finding. The issue only affects how wallets and marketplaces display the position, with no effect on transfers, lock state, shares or funds. The JetBit frontend reads position data directly instead of through the token URI.

[L-03] JetToken#_update - Transfer-lock gate does not guarantee the pre-TGE pool cannot trade

Description

`JetToken._update` enforces the pre-TGE transfer lock at the ERC-20 layer only: a transfer passes if trading is open, or if the sender or recipient is allowlisted. The `JetLiquidityLocker` header comment and the deploy script rely on a stronger claim, that this ERC-20 gate "blocks every swap" on the external PancakeSwap Infinity pool. That claim is unsound because the upstream Infinity Vault accounting separates currency deltas and internal credit/ownership from physical ERC-20 JET transfers. Pool accounting can change without triggering a JET `_update`, so the local assumption that no pre-TGE trade is possible is stronger than what the token hook actually enforces. Source identifier: `TP-TG-F08-POOL-ASSUMPTION`.

Vulnerability Details

The token gate checks only its own ERC-20 movement:

```
function _update(address from, address to, uint256 value) internal override {
    if (!tradingOpen && from != address(0) && to != address(0)) {
        if (!isAllowed[from] && !isAllowed[to]) revert TransfersLocked();
    }
    super._update(from, to, value);
}
```

This reverts a pre-TGE ERC-20 transfer unless one side is allowlisted. It fires only when JET tokens actually move as ERC-20 balances (`super._update`). It says nothing about state maintained inside an external venue that does not move ERC-20 JET on every economic action.

The locker's contract-level comment asserts the opposite as a safety property:

```
*  Pre-TGE the pool is untradeable (JetToken's transfer lock blocks every swap —
 *  only this locker and the presale are allowlisted), so no fees accrue and each
 *  add's delta is pure principal. `collectFees` exists for the post-TGE life of the
 *  position.
```

And `DeployPresale.s.sol` encodes the same assumption at wiring time:

```
// 4. Allowlist the presale + locker so their JET transfers pass the lock,
        //    then fund each allocation. The pool/traders are NEVER allowlisted, so
        //    the pre-seeded pool stays untradeable until openTrading() at TGE.
```

The PancakeSwap Infinity design (Vault + VaultToken) tracks per-currency deltas and internal credit balances during a `lock` session and settles at the end. A participant can hold or transfer internal currency credit, and pool accounting can be adjusted, through Vault operations (`take`, `settle`, `mint`/`burn` of internal credit) that do not each perform an ERC-20 `transfer` of JET. Only the net settlement touches ERC-20 balances, and even then only for the currency actually moved. So a swap-like reallocation of value inside the venue does not necessarily route through `JetToken._update`, which means the ERC-20 lock cannot be assumed to block every pool trade.

The 2026-09-09 validation follow-up corrects the delta-sign wording of the external report and this correction is preserved here: minting internal Vault credits applies a negative adjustment to the caller's delta and can offset a positive credit; it does not by itself settle an existing negative debt. In other words, internal credit operations can net against positive deltas but do not on their own discharge an outstanding amount owed to the Vault. The corrected mechanic still supports the finding: the locker's `_pay` only settles negative deltas via a real `safeTransfer` of the underlying token, while positive deltas are taken back, so the ERC-20 hook is exercised only on net settlement, not on intra-session accounting.

```
function _pay(Currency c, int128 amt) internal returns (uint256 paid) {
    if (amt < 0) {
        paid = uint256(uint128(-amt));
        VAULT.sync(c);
        IERC20(Currency.unwrap(c)).safeTransfer(address(VAULT), paid);
        VAULT.settle();
    } else if (amt > 0) {
        VAULT.take(c, address(this), uint256(uint128(amt)));
    }
}
```

Existing controls: the external accounting requires all lock deltas to settle and requires sufficient authorized balances, so this is not the creation of unbacked JET nor a bypass

of the token's own balance storage. The requirement that all lock deltas settle is preserved.

Limits: the external upstream source (Infinity `Vault.sol` / `VaultToken.sol`) was inspected on the review date and not matched byte-for-byte to the deployed Infinity contracts. The locker/token pool deployment and its live state are unresolved, and no mainnet-fork execution results were adopted, so a concrete pre-TGE trade was not demonstrated end-to-end.

Proof of Concept

This is an integration-correctness finding about an unsound assumption; the walkthrough shows why the ERC-20 gate does not cover internal Vault accounting.

Consider the two layers:

```
// Layer 1 — JetToken ERC-20 gate. Fires ONLY on an ERC-20 balance move.
function _update(address from, address to, uint256 value) internal override {
    if (!tradingOpen && from != address(0) && to != address(0)) {
        if (!isAllowed[from] && !isAllowed[to]) revert TransfersLocked(); // (A)
    }
    super._update(from, to, value);
}
```

Path 1 (covered): a plain `jet.transfer(pool, x)` pre-TGE with neither side allowlisted hits branch (A) and reverts. This is the only path the comment's "blocks every swap" reasoning accounts for.

Path 2 (not covered): inside an Infinity `lock` session, value is reallocated by adjusting currency deltas and internal credit. Numerically, suppose a caller holds internal credit for the JET currency and performs an operation that mints credit `+c` and offsets a positive delta:

```
caller JET delta before:  +c   (credited)
mint internal credit:     applies -c to caller delta  ->  net delta 0
```

No ERC-20 `transfer(from, to, value)` of JET occurs for that offset, so branch (A) is never reached; `JetToken._update` is not invoked. Settlement only calls `super._update` for

a currency with a remaining negative delta that is paid via `safeTransfer` (see `_pay`). Therefore economic activity that nets out inside the session, or is denominated in the counter-currency, does not trip the JET lock. The assumption "only this locker and the presale are allowlisted, so the pool stays untradeable" does not follow from the gate in `_update`.

Note the corrected sign semantics: the credit mint offsets a positive delta but does not settle a pre-existing negative debt, so this walkthrough does not claim creation of unbacked JET, only that pool accounting can move without an ERC-20 JET transfer.

Impact

Under the reviewed upstream accounting design, pool accounting and currency-denominated credit changes need not transfer ERC-20 JET, so the local assertion that the token gate blocks every pre-TGE pool trade is unsound. The concrete consequence is a design assumption baked into the locker comment and deploy wiring that overstates the guarantee: pre-TGE pool-price stability cannot be treated as automatically enforced by the ERC-20 lock. A realized monetary loss is not established, the delta-sign correction rules out unbacked-JET creation, and the external source was not matched to the deployed Infinity contracts; live pool deployment and state remain unresolved. The security outcome is therefore pending rather than a demonstrated live exploit.

Recommendation

Do not rely on the ERC-20 transfer lock to keep the external pool untradeable pre-TGE. Treat pre-TGE pool-price stability as an independent requirement and either avoid enabling a tradeable market until intended, or enforce a pool-level policy (e.g. a hooks contract on the pool key, or deferring liquidity/market enablement) rather than assuming the token gate covers it. Update the locker comment and deploy notes to state the real guarantee.

```
--- a/src/lp/JetLiquidityLocker.sol
+++ b/src/lp/JetLiquidityLocker.sol
@@
- *  Pre-TGE the pool is untradeable (JetToken's transfer lock blocks every swap –
```

```
- *  only this locker and the presale are allowlisted), so no fees accrue and each
- *  add's delta is pure principal. `collectFees` exists for the post-TGE life of the
- *  position.
+ *  Pre-TGE, JetToken's transfer lock blocks ERC-20 JET *transfers* by
non-allowlisted
+ *  parties. It does NOT by itself guarantee the external Infinity pool cannot
trade:
+ *  Vault accounting can move currency deltas / internal credit without an ERC-20
JET
+ *  transfer. Pre-TGE price stability must be enforced independently (e.g. a pool
+ *  hook, or not enabling a market) — do not assume the token gate covers every
swap.
```

Status

Acknowledged

Note

JetBit acknowledged this finding as a design clarification. JetBit does not rely on the ERC-20 transfer lock alone to keep the external pool untradeable before TGE, since a tradeable JET market is only enabled at TGE and the pool is held at the $1.00 reference until then. JetBit will correct the misleading code comment.

Centralisation

The JetBit project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the JetBit project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au

